bstorm

FORMATION

Formation n8n

Automatisation no-code / low-code — De la découverte à la mise en production

bstorm

SOMMAIRE

Les trois chapitres de la formation

01 Introduction à n8n et premiers workflows

02 Données, expressions et credentials

03 Logique, flux et robustesse

CHAPITRE 01

Introduction à n8n et premiers workflows

Comprendre la plateforme, la déployer, et construire ses premiers workflows automatisés.

OBJECTIFS PÉDAGOGIQUES

À la fin de ce chapitre, vous saurez...

01 Expliquer ce qu'est n8n et le situer face à Zapier et Make.

02 Distinguer Cloud et self-hosted, et comprendre le modèle de facturation.

03 Lancer une instance n8n locale via Docker.

04 Vous repérer dans l'interface : canvas, éditeur de node, exécution.

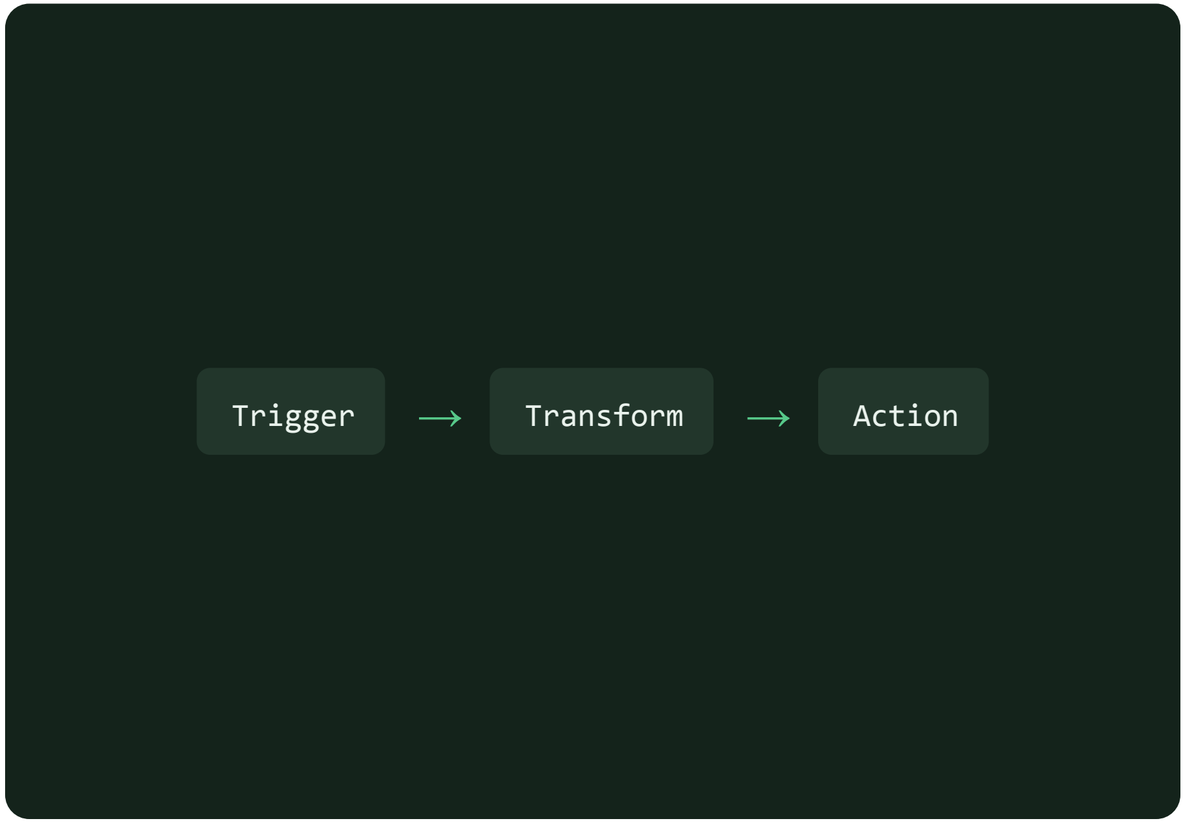
05 Décrire l'anatomie d'un workflow et les grandes familles de nodes.

06 Construire et exécuter un premier workflow simple.

QU'EST-CE QUE N8N ?

Une plateforme d'automatisation no-code / low-code, pilotée par un éditeur visuel.

Au lieu d'écrire un script qui va chercher des données, les transforme et les envoie ailleurs, on **assemble visuellement des blocs** qui font chacun une chose. Chaque étape est un **node** ; on relie les nodes entre eux pour décrire un traitement.



Un workflow est un **graphe orienté** : la donnée circule de node en node, de la gauche vers la droite.

CE QUI DISTINGUE N8N

Trois partis pris

Ouvert & auto-hébergeable

Le code est public. On fait tourner n8n sur sa propre infrastructure et on garde la maîtrise totale de ses données.

Extensible

Plus de 400 intégrations natives, et la possibilité d'écrire du JavaScript ou du Python directement dans un workflow.

Pensé pour l'IA

Des nodes dédiés permettent de construire de vrais agents IA — pas seulement « saupoudrer » de l'IA sur une automatisation.

« Faire de n8n **l'Excel de l'IA** » — assez puissant pour les développeurs, assez accessible pour les non-développeurs.

— Jan Oberhauser, fondateur

HISTORIQUE · L'ORIGINE

D'un studio d'effets visuels à un outil d'automatisation

Le fondateur **Jan Oberhauser** a travaillé dans les effets visuels pour le cinéma (*Maleficent*, *Happy Feet Two*) comme « pipeline TD » : automatiser le travail des artistes.

Son constat, qui deviendra la thèse de n8n : des gens très compétents restaient **dépendants d'ingénieurs** pour automatiser leurs tâches. Il commence à coder n8n en 2018.

LE NOM

n8n

Se prononce « n-eight-n ».

Numéronyme de **nodemation** (node + automation) : le 8 remplace les huit lettres — comme « k8s » pour Kubernetes.

HISTORIQUE · LE LANCEMENT

GitHub, octobre 2019 — et une licence « fair-code »

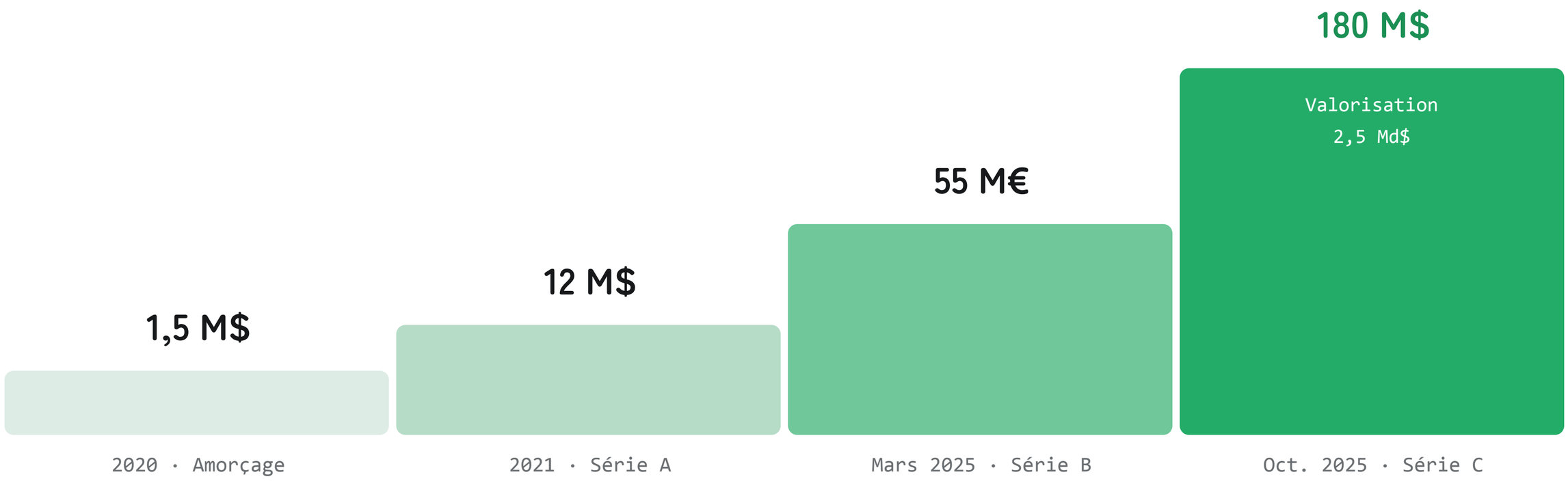
Première version publiée sur GitHub à Berlin. Dès le départ, une philosophie **fair-code** , plus tard formalisée par la **Sustainable Use License (SUL)**.

- ✓ Auto-héberger et modifier n8n pour un **usage interne** : autorisé.
- ✗ Revendre n8n **en tant que service** : restreint.

Un compromis entre l'open source permissif et le logiciel propriétaire, destiné à empêcher les grands hébergeurs cloud de revendre n8n **sans contribuer en retour**.

HISTORIQUE · LA CROISSANCE

Une trajectoire qui s'accélère avec l'IA



2026 : entrée de **SAP** au capital. Une communauté qui se compte désormais en **centaines de milliers** d'utilisateurs actifs.

HISTORIQUE · LE TOURNANT IA

Après ChatGPT, l'équipe reconstruit n8n pour en faire la couche d'orchestration des workflows d'IA.

Plutôt que de subir la menace, le produit est refondu en quelques semaines. C'est ce virage vers l'**IA agentique** qui a fait décoller la plateforme — aujourd'hui, une large majorité des workflows impliquent de l'IA.

MODES DE DÉPLOIEMENT

Trois formes, selon le besoin de contrôle

NOTRE CHOIX

Community self-hosted

Open source, hébergée soi-même (Docker, VPS…). Exécutions **illimitées** , toutes les intégrations, zéro licence.

En contrepartie : on gère serveur, base, mises à jour et sauvegardes.

MANAGÉ

n8n Cloud

n8n héberge tout : pas de serveur, mises à jour et montée en charge automatiques. Abonnement selon le volume.

Idéal pour démarrer vite ou sans compétence DevOps.

ORGANISATIONS

Business / Enterprise

SSO, contrôle d'accès avancé, gestion des environnements, versioning, journalisation.

En cloud ou self-hosted selon l'offre.

COMMUNITY VS CLOUD

Deux mondes en un coup d'œil

CRITÈRE	**Community (self-hosted)**	**Cloud**
Coût	Gratuit + infrastructure	Abonnement mensuel
Exécutions	**Illimitées**	Selon le plan
Données	Chez vous	Chez n8n
Maintenance	À votre charge	Gérée par n8n
Mise en route	~30–60 min	Immédiate

LE MODÈLE DE TARIFICATION · POINT CLÉ

n8n facture à l'exécution , pas à l'étape.

Une **exécution** = un run complet d'un workflow, du déclencheur jusqu'au bout — peu importe le nombre de nodes traversés.

`2 nodes = 1 exécution`

`50 nodes = 1 exécution`

→ la complexité est quasiment gratuite.

bstorm

FACE AUX CONCURRENTS

Un workflow de 10 étapes, facturé comment ?

Zapier

Facture à la tâche

10 tâches

Une tâche par étape, à chaque run.

Make

Facture à l'opération

10 opér.

Chaque module appelé compte.

n8n

Facture à l'exécution

1 exécution

10 étapes ou 50, c'est pareil.

Enrichir un lead → écrire au CRM → poster sur Slack → mettre à jour un tableur = **4 tâches ailleurs**, mais **1 seule exécution** ici.

LE PIÈGE À CONNAÎTRE · LE POLLING

Un déclencheur qui vérifie « toutes les 5 minutes »...

~8 640

exécutions par mois, **à lui seul** — qu'il trouve du nouveau ou non.

Sur un petit plan **Cloud** , il épuise le quota en quelques jours.

Sur **self-hosted** (notre cas), les exécutions sont illimitées : le sujet devient une question de **charge serveur** , pas de facture.

À retenir : on paie quand un workflow s'exécute de bout en bout — jamais au nombre de nodes ni de connecteurs.

LANCER SON INSTANCE

Environnement fourni, prêt à l'emploi

```
▸ http://localhost:5678
```

On y crée le **compte propriétaire local** . Aucune configuration demandée : l'objectif du jour est de construire des workflows, pas d'administrer le serveur.

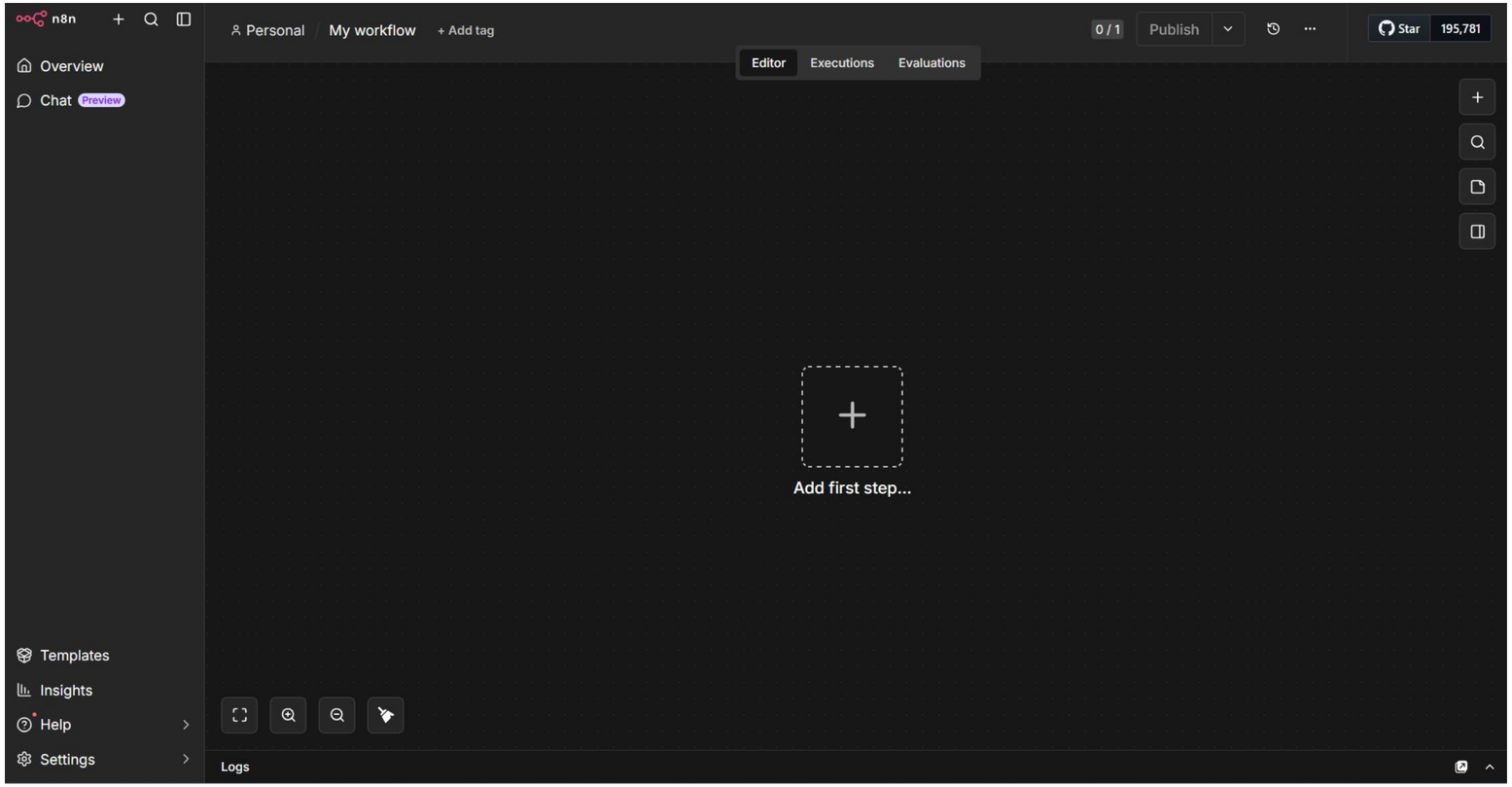
n8n 2.0 : le node Code s'exécute en **sandbox** par défaut, et un paradigme *Publish / Save* apparaît. Version de maturité — sans changement de fond sur la façon de construire.

TOUR DE L'INTERFACE

Trois zones pour être autonome

01 / Canvas

L'espace de travail où l'on dépose et relie les nodes. On ajoute un node avec le bouton **+** , puis on relie sortie et entrée du node suivant.

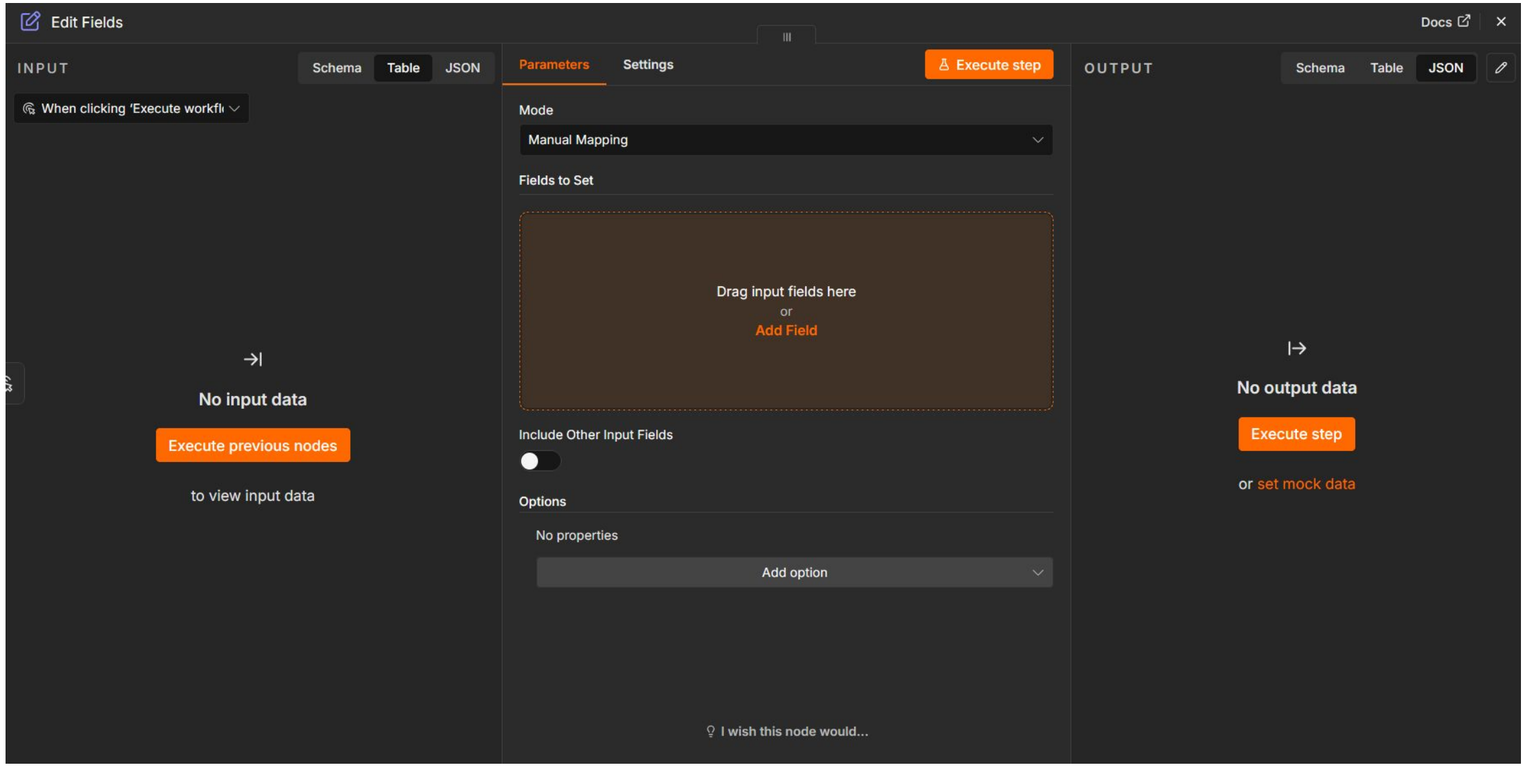


TOUR DE L'INTERFACE

Trois zones pour être autonome

02 / Éditeur de node

En double-cliquant : trois colonnes — **input** à gauche, **paramètres** au centre, **output** à droite. C'est ici qu'on passe le plus de temps.

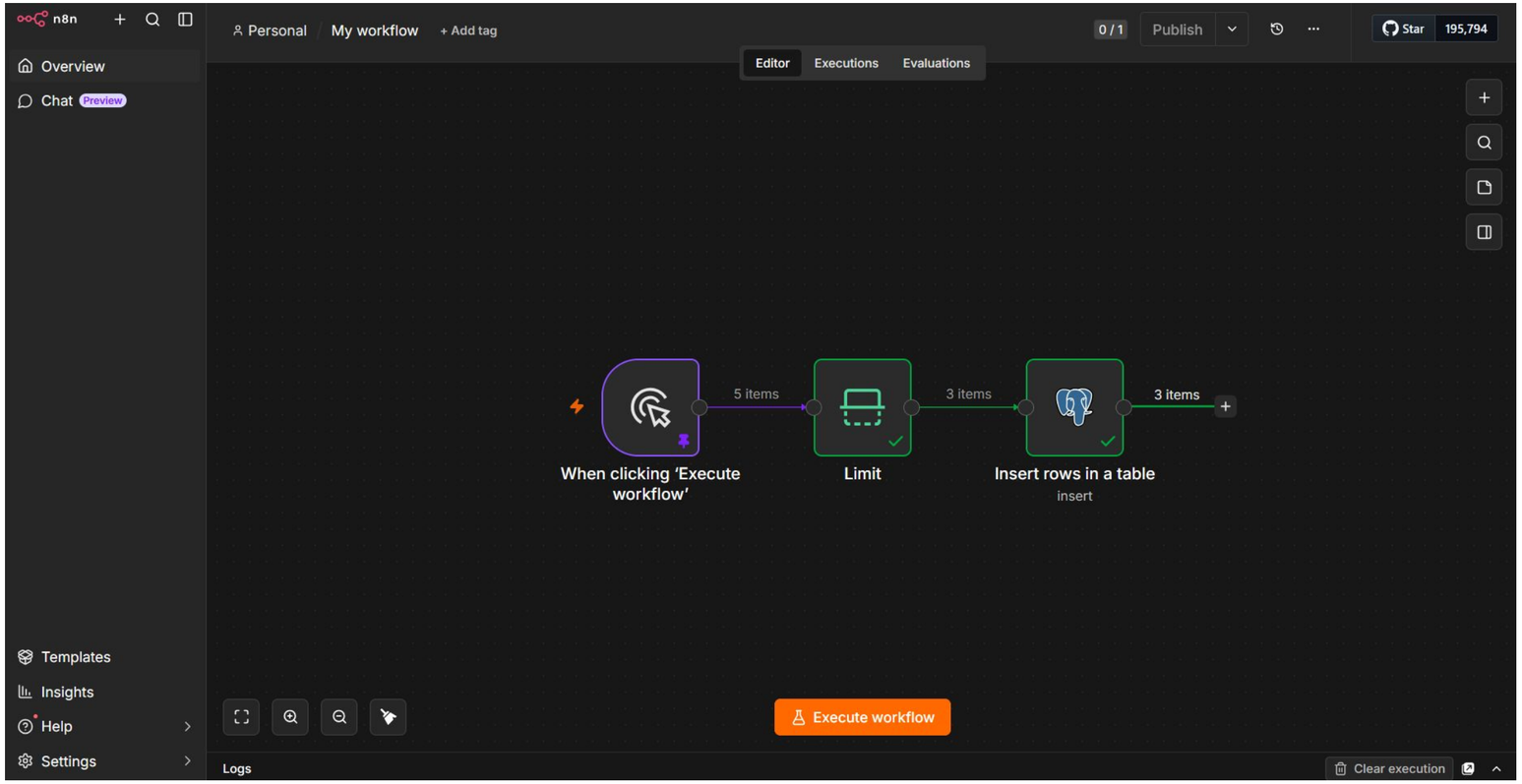


TOUR DE L'INTERFACE

Trois zones pour être autonome

03 / Exécution

Après un run, on visualise ce que chaque node a **reçu et renvoyé** . L'outil de débogage central : on suit la donnée node par node.

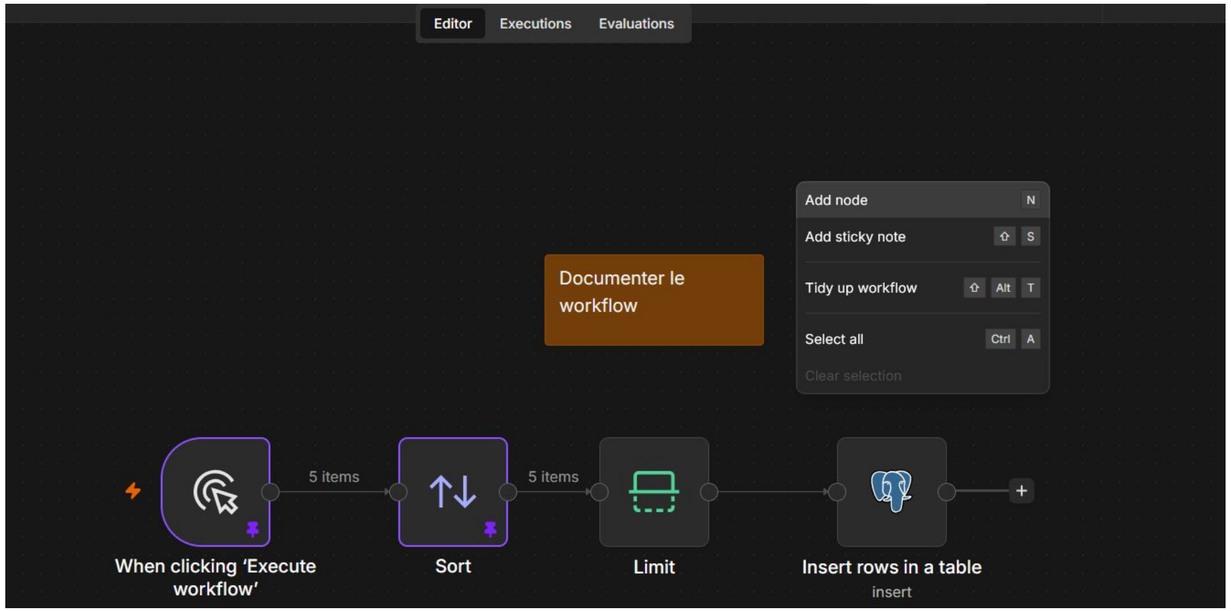


DOCUMENTER LE WORKFLOW.

Les Sticky Notes

Des post-it posés sur le canvas pour **documenter un workflow** . Une bonne habitude à prendre tout de suite.

Click droit sur le canvas -> Add sticky note



ANATOMIE D'UN WORKFLOW

Trois notions fondamentales

Le node	L'unité de base : un bloc qui fait une chose — déclencher, appeler une API, transformer, brancher. Un workflow est un assemblage de nodes.
La connexion	Le fil qui relie la sortie d'un node à l'entrée d'un autre. Il matérialise le sens de circulation : un workflow se lit **de gauche à droite**.
L'item	**Le point le plus important.** La donnée qui circule n'est pas « une valeur » : c'est une liste d'objets JSON. On y consacre la slide suivante.

L'ITEM · LE CONCEPT CENTRAL

La donnée circule en liste d'items JSON — un node s'exécute une fois par item.

```
[
  { "json": { "nom": "Dupont", "ville": "Bruxelles" }
},
  { "json": { "nom": "Martin", "ville": "Liège"     } }
]
```

Chaque node **reçoit une liste d'items en entrée et renvoie une liste en sortie** .

Ici, **deux items** : le node suivant s'exécutera **deux fois** , une par ligne.

TYPOLOGIE DES NODES

Trois grandes familles

Trigger nodes

Lancent le workflow — il commence toujours par un trigger.

- Manual — tester à la main
- Schedule — horaire (polling)
- Webhook — appel HTTP entrant
- Application — événement tiers

Action nodes

Font quelque chose vers un service.

Envoyer un message, créer un enregistrement, appeler une API. La plupart des intégrations : **Slack, Google Sheets, HTTP Request…**

Core nodes

Logique et manipulation, indépendants de tout service.

Edit Fields (Set) , IF / Switch, Merge, Code. Détaillés aux chapitres 2 et 3.

À retenir : tout workflow commence par un trigger, puis enchaîne des actions et de la logique.

CHAPITRE 1 · POINTS CLÉS À RETENIR

Ce qu'il faut garder en tête

01 Plateforme **node-based** , open source et auto-hébergeable, centrée sur l'IA agentique.

02 Deux mondes : **Community self-hosted** (gratuit, illimité) et **Cloud** (managé).

03 Facturation **à l'exécution**, jamais à l'étape — attention au **polling**.

04 Un workflow **commence toujours par un trigger** , puis actions et logique.

05 La donnée circule en **liste d'items JSON** : un node s'exécute une fois par item.

06 Outils de survie : **panneau d'exécution**, **sticky notes**.

CHAPITRE 02

Données, expressions et credentials

Manipuler la donnée avec précision, écrire des expressions, et connecter des services authentifiés en toute sécurité.

OBJECTIFS PÉDAGOGIQUES

À la fin de ce chapitre, vous saurez...

01 Expliquer la structure de données de n8n et son impact sur l'exécution.

02 Composer et transformer des données avec Edit Fields (Set).

03 Utiliser le pinning pour figer des données de test et déboguer plus vite.

04 Écrire des expressions et connaître les variables intégrées.

05 Déboguer une expression avec l'éditeur et l'aperçu en direct.

06 Créer, réutiliser et sécuriser des credentials ; clé d'API vs OAuth2.

Dans n8n, tout circule sous forme d'Items.

Un workflow ne manipule pas directement des variables ou des tableaux : il traite une **liste d'items**.

Chaque nœud suit toujours le même cycle en trois temps.

01 Reçoit une **liste d'items** en entrée

02 Effectue un **traitement**

03 Renvoie une **nouvelle liste d'items**

À retenir : dans n8n, on pense toujours en termes d'items.

QU'EST-CE QU'UN ITEM ? · DÉFINITION

Un Item représente une unité de données indépendante.

Le plus souvent, un item correspond à :

- un client
- une commande
- un email
- un fichier
- une ligne Excel
- un enregistrement BDD

Chaque item contient au minimum un objet **JSON**.

```
{
  "json": {
    "nom": "Dupont",
    "email": "d@exemple.be",
    "ville": "Bruxelles"
  }
}
```

PLUSIEURS ITEMS

Un workflow traite généralement plusieurs items simultanément.

```
[
  { "nom": "Alice",   "ville": "Paris"     },
  { "nom": "Bob",     "ville": "Lyon"      },
  { "nom": "Charlie", "ville": "Bruxelles" }
]
```

Ici, le workflow reçoit **3 items**.

Chaque item sera traité **indépendamment** par les nœuds.

Le node suivant s'exécutera donc **trois fois**, une fois par item.

DE QUOI EST COMPOSÉ UN ITEM ?

Un item possède plusieurs propriétés.

Un item ne contient pas uniquement des données : il peut aussi transporter des **fichiers** ou des **informations techniques**.

> Dans **95 %** des workflows, vous manipulerez principalement la propriété `json`.

```
Item
├── json
├── binary        (optionnel)
├── pairedItem    (interne)
└── ...
```

LA PROPRIÉTÉ JSON · LES DONNÉES MÉTIER

La propriété json contient les informations que votre workflow traite.

```
{
  "json": {
    "prenom": "Alice",
    "nom": "Martin",
    "email": "alice@mail.com"
  }
}
```

Dans les expressions :

```
{{ $json.prenom }}
```

```
{{ $json.email }}
```

C'est la propriété **la plus utilisée** dans n8n.

Les documents manipulés sont stockés dans binary.



On retrouve cette propriété avec :

- PDF
- Images
- Audio
- Vidéo
- Archives ZIP
- Documents Word

JSON

contient les **informations**

BINARY

contient le **fichier lui-même**

EXEMPLE · JSON + BINARY

Un workflow reçoit un CV.



Le même item transporte :

- les informations sur le **candidat** ;
- le document **PDF**.

Les deux restent **associés** tout au long du workflow.

https://docs.n8n.io/build/work-with-data/understand-n8ns-data-structure

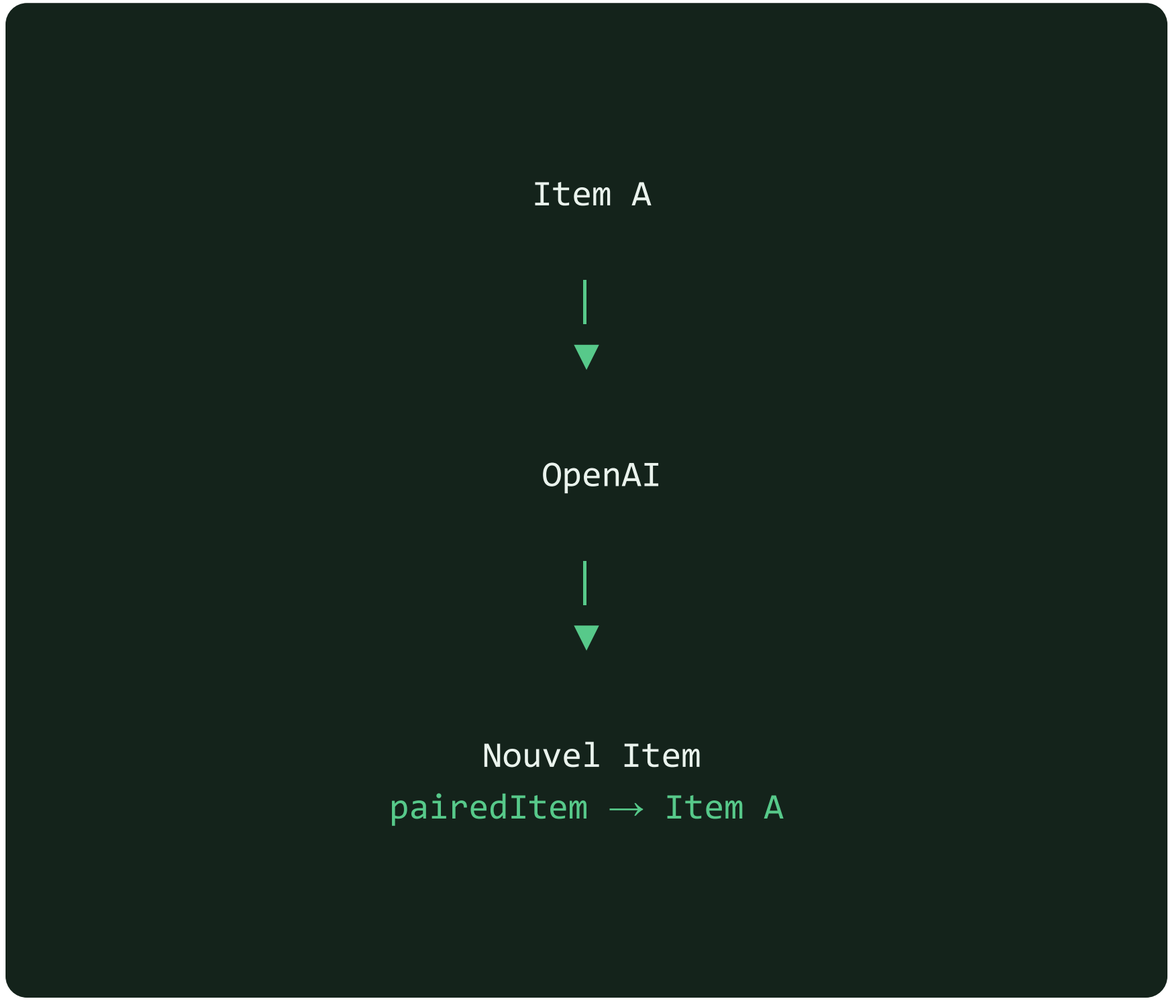
LA PROPRIÉTÉ PAIREDITEM · LE LIEN ENTRE LES ITEMS

n8n garde un lien vers l'item d'origine avec pairedItem.

Certains nœuds créent de nouveaux items. Cette propriété permet notamment :

- de retrouver la **source** d'un item ;
- de faciliter le **débogage** ;
- d'assurer la **traçabilité** des données.

Cette propriété est principalement utilisée par n8n **en interne**.



LES MÉTADONNÉES INTERNES

n8n ajoute parfois des informations techniques.

`pairedItem`

le lien vers l'item d'origine.

`exécution`

les métadonnées d'exécution.

`liaison`

les informations de liaison entre les nœuds.

Ces propriétés sont **gérées automatiquement** — dans la majorité des workflows, vous n'aurez pas à les modifier.

LE NŒUD EDIT FIELDS · MODIFIER LES DONNÉES D'UN WORKFLOW

Modifier les données contenues dans les items.

Il peut notamment :

- \+ Ajouter de nouveaux champs
- ✎ Modifier des valeurs existantes
- × Supprimer des champs
- ⇄ Renommer des champs
- Sélectionner les données à conserver

Edit Fields

C'est le nœud de **transformation de données** le plus utilisé dans n8n.

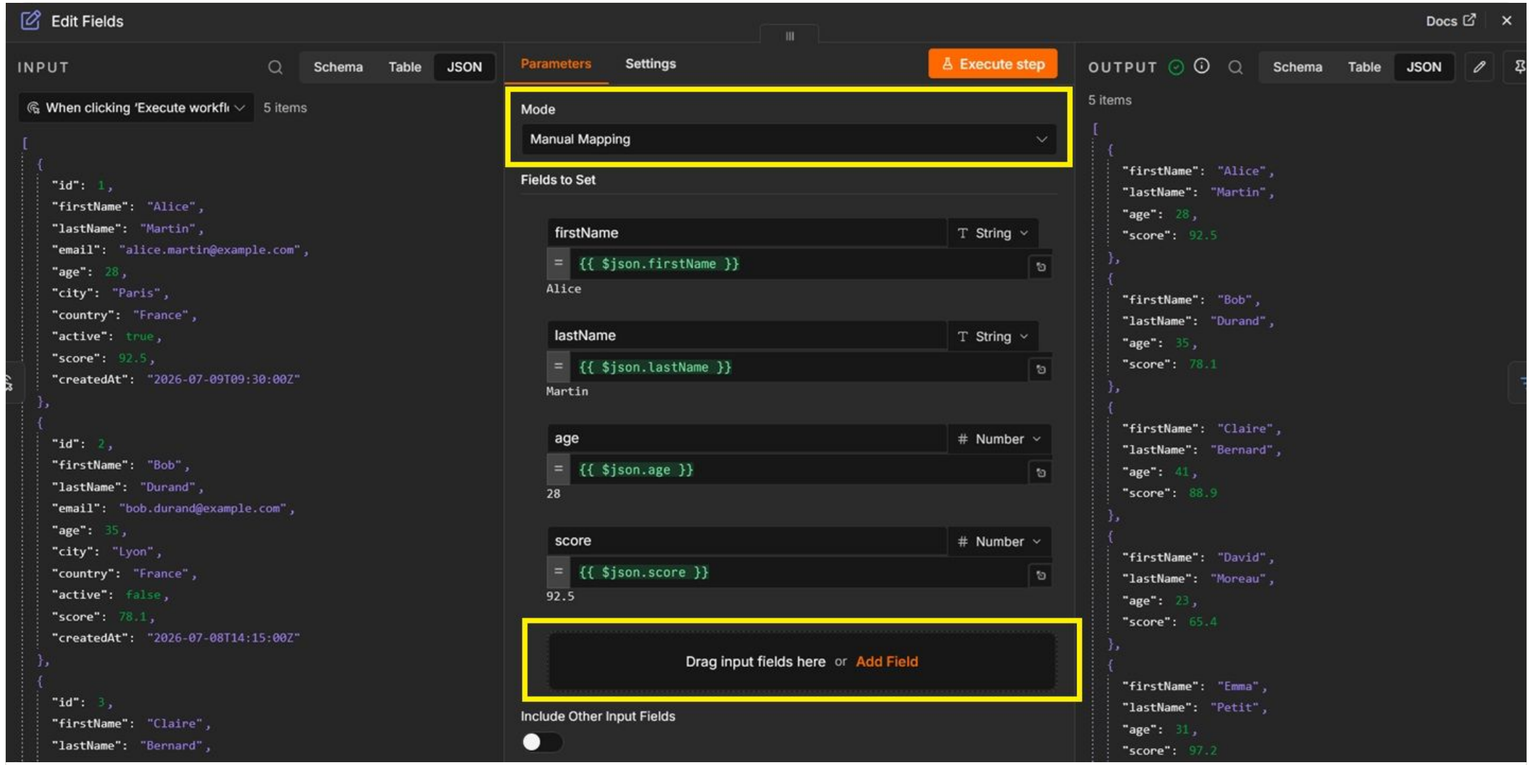
CAS D'USAGE

Le nœud Edit Fields est souvent utilisé pour :

- Préparer un **prompt** pour un LLM
- Construire le corps d'une **requête HTTP**
- Renommer des **colonnes**
- Ajouter un **identifiant**
- Formater des données avant une **insertion en base**
- Nettoyer les données avant un **export**

C'est généralement le **premier nœud de transformation** dans un workflow.

Mapping mode & Drag and drop fields.



Mapping mode:

- Manual : permet de drag and drop les champs depuis l'input.
- Json : permet d'écrire le json directement.

Drag and Drop:

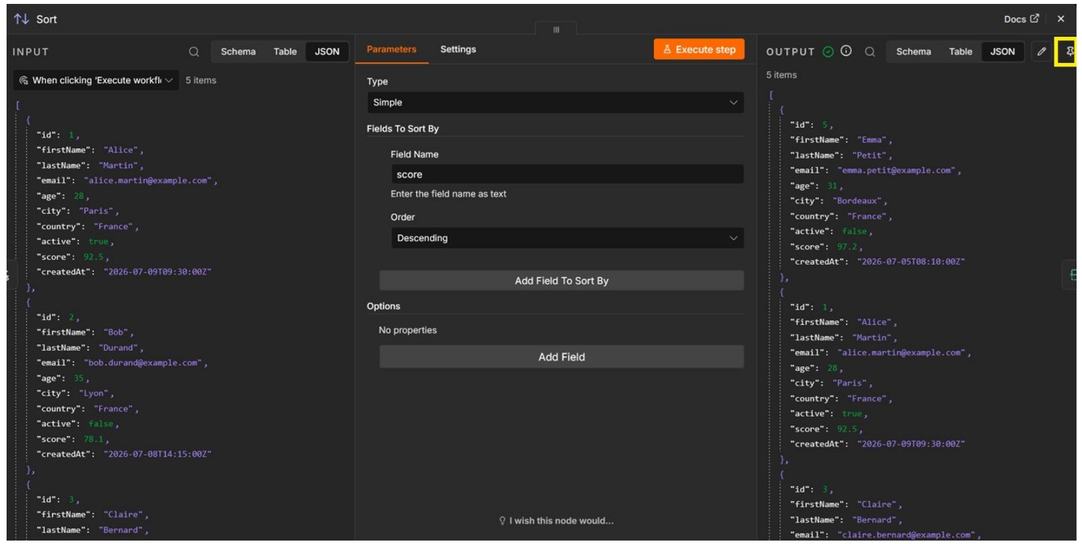
Espace pour drag and drop les champs depuis l'input ou rajouter de nouveaux champs

LE PINNING DES DONNÉES DE TEST

Figer la sortie d'un node pour itérer sans réexécuter

Un node « épinglé » ne se réexécute pas : n8n réutilise les **données gelées** de la dernière exécution de test.

- Éviter de rappeler une API lente, payante ou à quota.
- Travailler sur des données stables en aval.
- Reproduire un cas précis pour le corriger.



EN PRATIQUE

Exécuter une fois → **épingler** le node source (icône punaise 📌) → itérer sur la suite autant qu'on veut → dépingler pour repasser en conditions réelles.

EXERCICES · CHAPITRE 1

Exercice 1.1 — Une plateforme opérationnelle

LE BESOIN

> *Une PME vient d'installer une plateforme d'automatisation et souhaite vérifier qu'elle est opérationnelle. Elle demande un premier workflow qu'un collaborateur peut lancer lui-même, à la demande, et dont on peut consulter le résultat.*

RESSOURCES

Instance n8n : `http://localhost:5678`

RÉSULTAT ATTENDU

- ✓ Un compte propriétaire est créé.
- ✓ Une exécution réussie est visible dans l'historique.
- ✓ Le résultat de l'exécution est consultable.

Objectif : prendre en main l'interface et le cycle exécuter → observer.

EXERCICES · CHAPITRE 1

Exercice 1.2 — L'arrivée des nouveaux collaborateurs

LE BESOIN

> *Un service RH prépare l'automatisation de l'accueil des nouveaux collaborateurs, mais son logiciel RH n'est pas encore connecté. Il souhaite disposer de données de test simulant l'arrivée de 3 personnes (prénom, nom, service, date d'arrivée) pour construire la suite du workflow.*

RESSOURCES

Aucune source externe : les données de test sont à créer dans le workflow.

RÉSULTAT ATTENDU

- ✓ La sortie contient 3 items distincts.
- ✓ Chaque item contient les 4 champs : prénom, nom, service, date d'arrivée.

Objectif : comprendre qu'un workflow manipule une liste d'items structurés.

EXERCICES · CHAPITRE 1

Exercice 1.3 — La météo du chantier

LE BESOIN

> *Une entreprise de construction envoie chaque matin ses équipes sur chantier à Bruxelles. Le chef de chantier souhaite connaître, en un clic, la température actuelle. À terme, cette vérification doit se lancer toute seule chaque matin à 6h30.*

RESSOURCES

API météo Open-Meteo (ouverte, sans clé) : `https://api.open-meteo.com/v1/forecast?latitude=50.85&longitude=4.35&current=temperature_2m`

RÉSULTAT ATTENDU

- ✓ La température actuelle à Bruxelles apparaît dans la sortie du workflow.
- ✓ Bonus : le workflow se déclenche automatiquement chaque matin à 6h30.

Objectif : faire entrer dans un workflow de la donnée provenant d'un service externe.

EXERCICES · CHAPITRE 1

Exercice 1.4 — Le fichier clients du service commercial

LE BESOIN

> *Le service commercial d'une société dispose d'un système qui renvoie des fiches clients très détaillées. Il souhaite n'en conserver que le nom, l'email, le téléphone et la ville de chaque client. Contrainte : chaque appel à ce système est facturé ; il ne doit pas être interrogé à chaque essai pendant la mise au point.*

RESSOURCES

Système clients (API ouverte, sans clé) : `https://jsonplaceholder.typicode.com/users`

RÉSULTAT ATTENDU

- ✓ La sortie contient 10 items, un par client.
- ✓ Chaque item contient exactement 4 champs clairement nommés : nom, email, téléphone, ville.
- ✓ Le système n'est interrogé qu'une seule fois pendant la mise au point.

Objectif : réduire une donnée riche à l'essentiel, et la mettre au point sans rappeler le service.

EXERCICES · CHAPITRE 1

Exercice 1.5 — Défi : la fiche produit du catalogue

LE BESOIN

> *Un e-commerçant souhaite générer une fiche produit simplifiée pour son catalogue : nom du produit, catégorie, prix, stock disponible, note moyenne et image. Il faut commencer par le produit n°1. Le workflow sera ensuite repris par un collègue, qui doit pouvoir le comprendre et l'adapter à un autre produit sans aide.*

RESSOURCES

Catalogue produits (API ouverte, sans clé) : `https://dummyjson.com/products/1`

RÉSULTAT ATTENDU

- ✓ La sortie contient 1 item avec uniquement les 6 champs demandés, clairement nommés.
- ✓ Les nodes portent des noms explicites et une note sur le canvas explique le workflow.
- ✓ Changer de produit ne nécessite de modifier qu'un seul endroit.

Objectif : construire un workflow complet, lisible et maintenable par un tiers.

LES EXPRESSIONS · RENDRE UN WORKFLOW DYNAMIQUE

Récupérer ou calculer une valeur à partir des données du workflow.

SANS EXPRESSION

`alice@email.com`

Une valeur fixe, identique pour tous les items.

→

AVEC UNE EXPRESSION

`{{$json.email}}`

Une valeur dynamique, lue sur l'item.

Une expression **adapte automatiquement sa valeur** en fonction de l'item en cours de traitement.

LA SYNTAXE

Toutes les expressions s'écrivent entre doubles accolades.

```
{{ ... }}
```

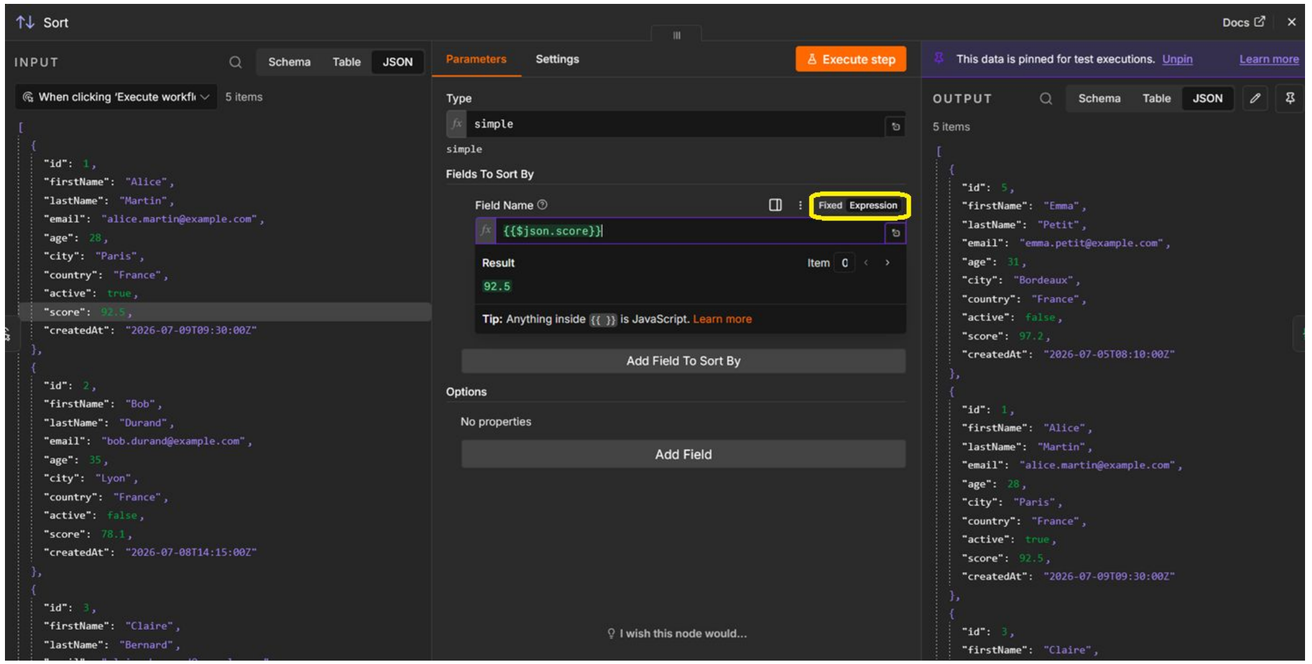
Tout ce qui se trouve entre les accolades est **évalué par n8n**.

EXEMPLES

```
{{$json.nom}}
```

```
{{$json.email}}
```

```
{{$json.age + 1}}
```



ACCÉDER AUX DONNÉES D'UN ITEM

Les données métier sont dans la propriété json.

ITEM

```
{
  "nom": "Alice",
  "email": "alice@email.com"
}
```

EXPRESSIONS

```
{{$json.nom}}
```

```
{{$json.email}}
```

`$json` représente les données de l'item **actuellement traité**.

UNE EXPRESSION ÉVALUÉE POUR CHAQUE ITEM

La même expression, une valeur par item.

ENTRÉE · ITEMS

- Alice
- Bob
- Charlie

`{{$json.nom}}`

→

SORTIE · VALEURS

- **Alice**
- **Bob**
- **Charlie**

Pas besoin d'écrire une **boucle** : n8n évalue l'expression pour chaque item.

LES EXPRESSIONS UTILISENT JAVASCRIPT

Lire les données, mais aussi les transformer.

Mettre en majuscules

```
{{$json.nom.toUpperCase()}}
```

Concaténer

```
{{$json.prenom + " " + $json.nom}}
```

Tester une condition

```
{{$json.age >= 18}}
```

Les expressions ne servent pas uniquement à **lire** des données — elles permettent aussi de les **transformer**.

Les **méthodes JavaScript** les plus courantes peuvent être utilisées directement.

LES VARIABLES LES PLUS UTILISÉES

Un vocabulaire de base

VARIABLE	DESCRIPTION
`$json`	Données de l'item courant
`$binary`	Fichiers de l'item courant
`$input`	Données reçues par le nœud
`$now`	Date et heure actuelles
`$execution`	Informations sur l'exécution

LES FONCTIONS INTÉGRÉES

En plus de JavaScript, n8n fournit des fonctions utiles.

Date au format ISO

```
{{$now.toISO()}}
```

Ajouter 7 jours

```
{{$now.plus({days:7})}}
```

Normaliser du texte

```
{{$json.email.toLowerCase()}}
```

`Dates` `Texte` `Nombres`

Ces fonctions facilitent la manipulation des **dates**, du **texte** et des **nombres**.

BONNES PRATIQUES

Quatre réflexes à adopter

- ✔ Utiliser les expressions **uniquement lorsque la valeur doit être dynamique** .
- ✔ Préférer des expressions **simples et lisibles**.
- ✔ Tester les expressions avec les **données d'exemple** du workflow.
- ✔ Éviter les calculs complexes : utiliser un **Code Node** si la logique devient importante.

LE BRANCHEMENT CONDITIONNEL

Trois nodes, trois usages distincts

IF

Deux chemins

Route chaque item vers **true** ou **false** selon des conditions typées. Le branchement binaire de base.

Switch

Plusieurs chemins

Route vers **plusieurs sorties** selon des règles, avec une sortie **fallback** . Pour aiguiller par catégorie ou statut.

Filter

Un tamis

Pas de deuxième chemin : **supprime** les items qui ne remplissent pas la condition. Pour réduire un jeu de données.

Règle de choix : deux issues à traiter → IF · plusieurs catégories → Switch · juste écarter des items → Filter.

LE NŒUD IF · EXÉCUTER UNE CONDITION

Tester une condition sur chaque item.

Selon le résultat, l'item est envoyé vers l'une des **deux sorties** .

`True`	la condition est **vraie**.
`False`	la condition est **fausse**.

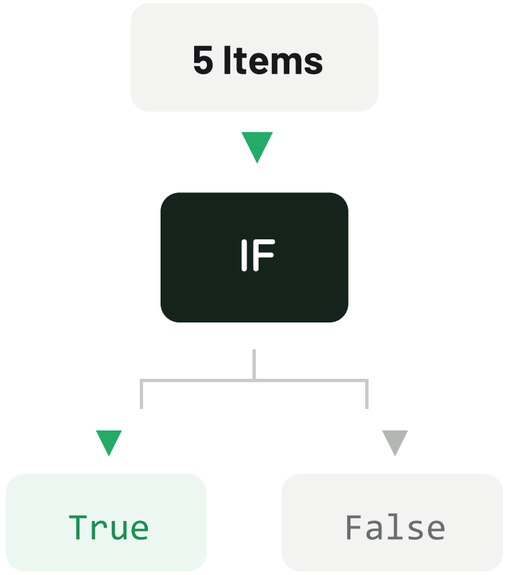
IF

Le nœud IF

Il permet d'ajouter de la **logique** à un workflow.

FONCTIONNEMENT

Le nœud analyse chaque item indépendamment.

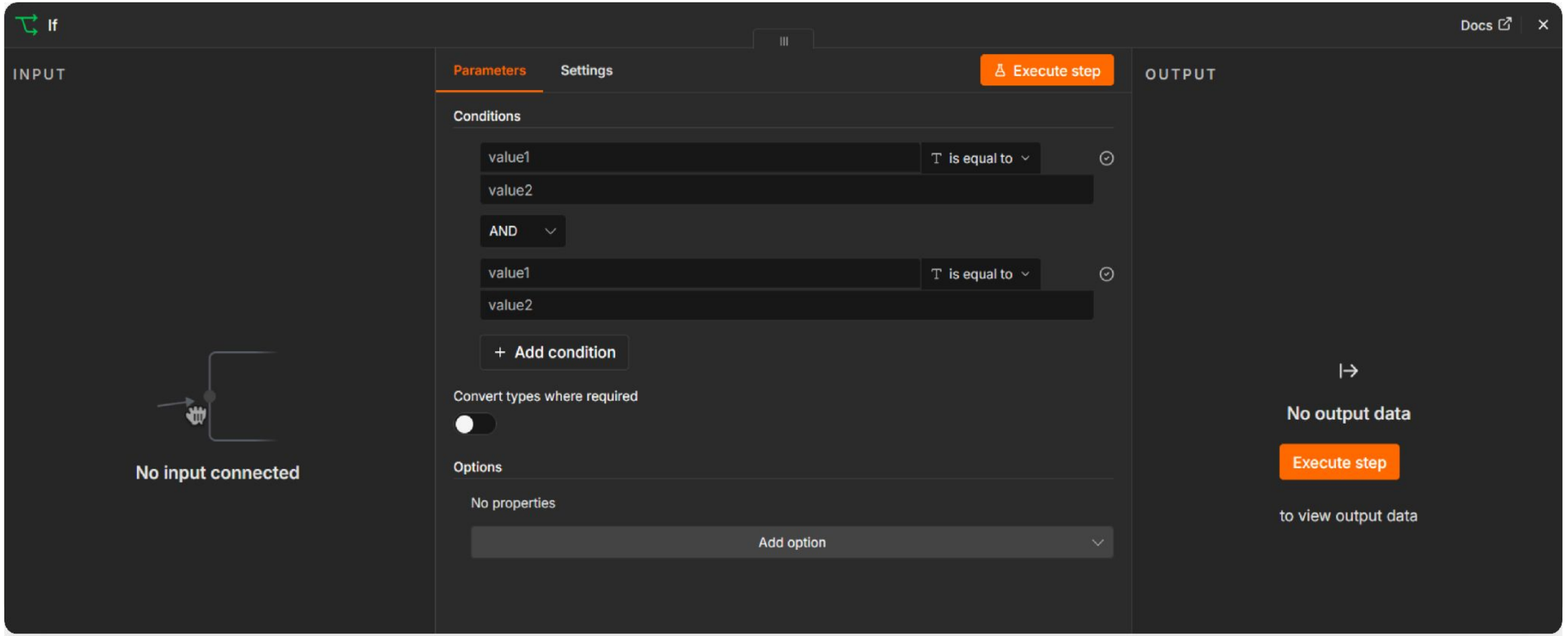


Chaque item est évalué **individuellement**.

Le nœud ne renvoie pas un résultat global : il **aiguille chaque item** vers la bonne sortie.

L'INTERFACE DU NŒUD

On définit une ou plusieurs conditions.



CONDITIONS
CONDITIONS
Comparer une **valeur** à une autre via un **opérateur**.
Comparer une **valeur** à une autre via un **opérateur**.

AND / OR
AND / OR
Cumuler plusieurs conditions avec **Add condition**.
Cumuler plusieurs conditions avec **Add condition**.

OUTPUT
OUTPUT
Les items sortent par **True** ou **False**.
Les items sortent par **True** ou **False**.

LE NŒUD SWITCH · DIRIGER VERS PLUSIEURS CHEMINS

Répartir les items selon plusieurs conditions.

NŒUD IF

Deux sorties seulement — **True / False**.

NŒUD SWITCH

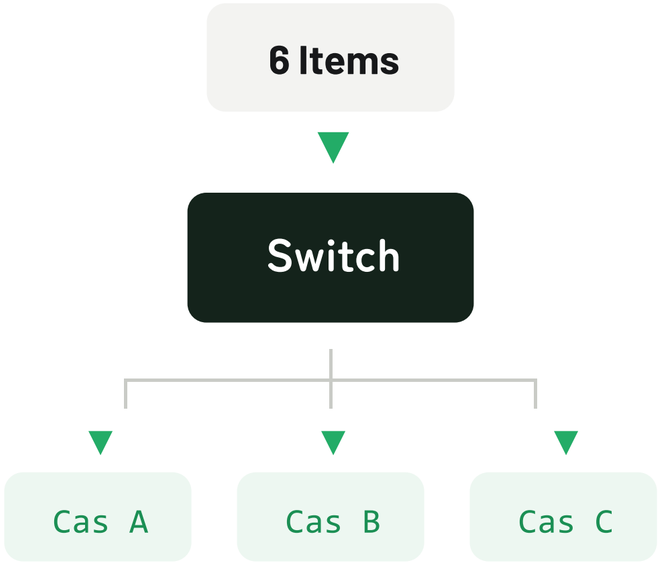
Autant de **cas** que nécessaire.

Switch

Idéal lorsque **plusieurs traitements** sont possibles.

FONCTIONNEMENT

Chaque item est évalué individuellement.



Chaque item part vers la sortie de la **première règle satisfaite** .

L'**ordre des règles** compte donc : la première qui correspond l'emporte.

LE NŒUD FILTER · CONSERVER LES ITEMS SOUHAITÉS

Filtrer une collection d'items selon des conditions.

✔ Les items qui satisfont les conditions sont **conservés**.

× Les autres sont **supprimés** du workflow.

Filter

Le Filter sert à **réduire** une collection d'items.

FONCTIONNEMENT

Chaque item est évalué indépendamment.



Une seule sortie : seuls les items **retenus** continuent.

Les items ne correspondant pas aux critères **ne poursuivent pas** leur parcours.

LE NŒUD HTTP REQUEST · COMMUNIQUER AVEC UNE API

Envoyer des requêtes HTTP vers un service externe.

Il est utilisé pour :

- Récupérer des données
- Envoyer des informations
- Mettre à jour des ressources
- Supprimer des données

⇄ **HTTP Request**

n8n peut communiquer avec pratiquement **n'importe quelle API** .

LES ÉLÉMENTS D'UNE REQUÊTE

Une requête HTTP est composée de plusieurs éléments.

URL

Adresse de la **ressource**.

MÉTHODE

Type d'action — `GET, POST, PUT, DELETE...`

HEADERS

Informations complémentaires — **authentification, format**.

BODY

Données envoyées avec la requête **(selon la méthode)**.

Tous les paramètres peuvent être **statiques** ou construits avec des **expressions**.

LES PRINCIPALES MÉTHODES HTTP

Cinq méthodes à connaître

MÉTHODE	UTILISATION
GET	Lire ou récupérer des données
POST	Créer une nouvelle ressource
PUT	Remplacer une ressource existante
PATCH	Modifier partiellement une ressource
DELETE	Supprimer une ressource

Le choix de la méthode dépend de l'**API utilisée**.

LES CREDENTIALS

S'authentifier, séparément des workflows

Chiffré au repos

Chiffrés avec la clé de l'instance. Jamais stockés en clair dans le workflow.

Séparés du workflow

Le workflow référence une credential par son **nom** ; le secret vit ailleurs. On partage sans divulguer.

Réutilisables

Une même credential sert dans plusieurs workflows. Mise à jour à **un seul endroit** .

LA CLÉ DE CHIFFREMENT DE L'INSTANCE · À QUOI SERT-ELLE ?

Chaque instance possède une clé de chiffrement.

Elle chiffre les **données sensibles** stockées dans la base de données, notamment :

- Les identifiants API
- Les mots de passe
- Les jetons d'accès (`Access Tokens`)
- Les Credentials utilisés par les workflows

ENCRYPTION KEY

••••••••••••••••

Sans cette clé, les informations sensibles **ne peuvent pas être déchiffrées** .

BONNES PRATIQUES · PROTÉGER SA CLÉ

La clé est propre à chaque instance.

- Définir explicitement la variable `N8N_ENCRYPTION_KEY`.
- Conserver une copie dans un endroit **sécurisé**.
- Utiliser la **même clé** lors d'une migration ou restauration.
- Ne **jamais partager** cette clé publiquement.

⚠ ATTENTION

Si vous **perdez cette clé** , les Credentials déjà enregistrés ne pourront **plus être déchiffrés** . Ils devront être recréés manuellement.

S'authentifier auprès de services externes.

Ils contiennent les informations nécessaires pour établir une **connexion sécurisée** avec une application ou une API.

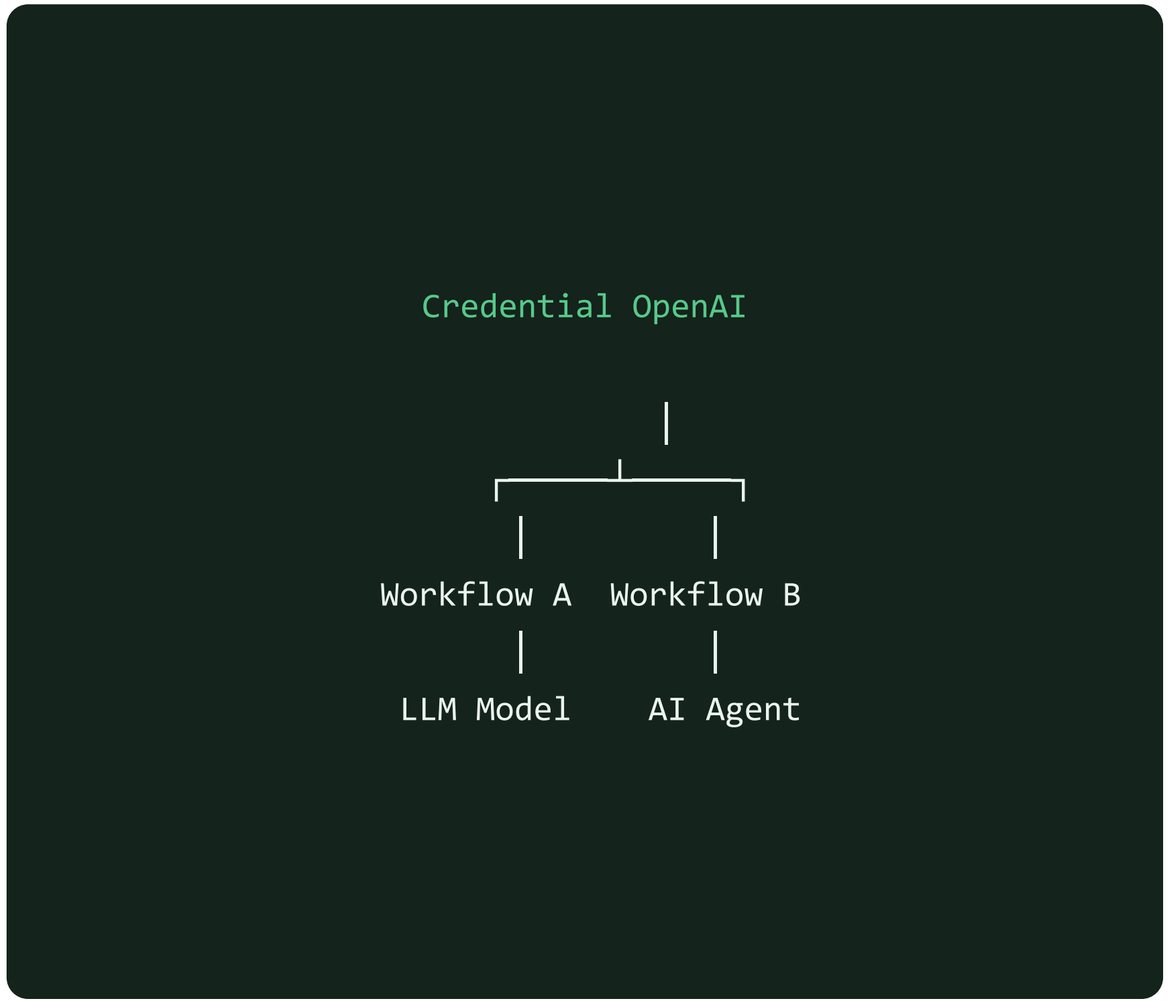
Les Credentials évitent de **saisir vos identifiants dans chaque nœud** .

EXEMPLES

- Clé API LLM Model
- Compte Google (OAuth2)
- Jeton GitHub
- Base Airtable
- Base de données PostgreSQL

RÉUTILISER LES CONNEXIONS · UN CREDENTIAL, PLUSIEURS WORKFLOWS

Un même Credential, plusieurs usages.



LES AVANTAGES

- Une seule configuration
- Maintenance simplifiée
- Mise à jour centralisée
- Moins de risques d'erreur

LES PRINCIPAUX TYPES D'AUTHENTIFICATION

Un mode adapté à chaque service.

API Key

Une clé unique par service.

OAuth2

Autorisation déléguée (Google…).

Basic Auth

Identifiant / mot de passe.

Bearer Token

Jeton transmis dans l'en-tête.

JWT

Jeton signé — plus rare.

n8n adapte **automatiquement** le formulaire selon le service.

SÉCURITÉ DES CREDENTIALS

Des informations sensibles, protégées automatiquement.

- Chiffrement **avant stockage** en base de données
- Utilisation de la **clé de chiffrement** de l'instance
- Valeurs **masquées** dans l'interface

⚠ ATTENTION

Ne stockez **jamais** vos clés API directement dans un workflow ou dans un nœud `Set`.

Privilégiez toujours les **Credentials**.

CLÉ D'API OU OAUTH2 ?

Choisir, puis prendre de bonnes habitudes

Clé d'API

Accès machine simple, service interne, script.

OAuth2

Quand le service l'impose, qu'on agit **au nom d'un utilisateur** , ou qu'on veut des jetons à durée limitée.

BONNES PRATIQUES

Nommer clairement (service + compte + environnement) · jamais de secret en clair dans un champ de node · **tester la connexion** avant de construire.

CHAPITRE 2 · POINTS CLÉS À RETENIR

Ce qu'il faut garder en tête

01 La donnée est une **liste d'items** ; chaque item a une clé `json` ; un node = une exécution par item.

02 **Edit Fields (Set)** compose et nettoie ; « Keep Only Set » produit une sortie propre.

03 Le **pinning** fige les données de test : moins d'appels API, débogage plus rapide.

04 Les **expressions** `{{}}` rendent les champs dynamiques ; toujours partir d'une variable.

05 L'**aperçu en direct** et le défilement des items sont les outils clés de débogage.

06 Les **credentials** sont chiffrées, séparées et réutilisables ; API pour le simple, OAuth2 si imposé.

Prochain chapitre : la logique et le contrôle de flux — IF, Switch, Merge, boucles, Code node, gestion d'erreurs et sous-workflows.

EXERCICES · CHAPITRE 2

Exercice 2.1 — Nettoyer une réponse d'API

1. Appeler une API publique renvoyant de nombreux champs (HTTP Request).
2. Avec **Edit Fields**, ne garder que 3 champs bien nommés (activer `Keep Only Set`).
3. Vérifier que la sortie ne contient plus que ces trois champs.

Objectif : maîtriser Edit Fields et « Keep Only Set ».

EXERCICES · CHAPITRE 2

Exercice 2.2 — Premières expressions

1. À partir d'un item contenant `prenom`, `nom`, `age` : composer un champ `nom_complet` en majuscules.
2. Créer un champ `categorie` = « adulte » ou « mineur » selon l'âge (ternaire).
3. Ajouter un champ `echeance` = aujourd'hui + 7 jours, formaté (Luxon).

Objectif : écrire des expressions et enchaîner des méthodes.

EXERCICES · CHAPITRE 2

Exercice 2.3 — Première credential

1. Créer une credential de type **Header Auth** (ou clé d'API) pour un service au choix.
2. La nommer clairement : `service · compte · env`, puis tester la connexion.
3. L'utiliser dans un node, exécuter, et confirmer qu'aucun secret n'apparaît en clair.

Objectif : créer et sécuriser une credential réutilisable.

EXERCICES · CHAPITRE 2

Exercice 2.4 — Défi : de la donnée brute au message

1. HTTP Request (avec credential) → épingler sa sortie.
2. Edit Fields : construire un champ `message` mêlant texte figé et expressions.
3. Déboguer via l'aperçu en direct, en faisant défiler les items.
4. Dépingler et réexécuter en conditions réelles.

Objectif : combiner expressions, pinning et credential de bout en bout.

CHAPITRE 03

Logique, flux et robustesse

Contrôler le flux, écrire de la logique sur-mesure, rendre un workflow robuste et le découper en briques réutilisables.

OBJECTIFS PÉDAGOGIQUES

À la fin de ce chapitre, vous saurez...

01 Brancher un workflow selon des conditions avec IF, Switch et Filter.

02 Recombiner des flux parallèles avec Merge et choisir le bon mode.

03 Traiter de gros volumes par lots avec Loop Over Items.

04 Écrire de la logique personnalisée dans le Code node (JS ou Python).

05 Rendre un workflow résilient : retry, sorties d'erreur, workflow d'erreur.

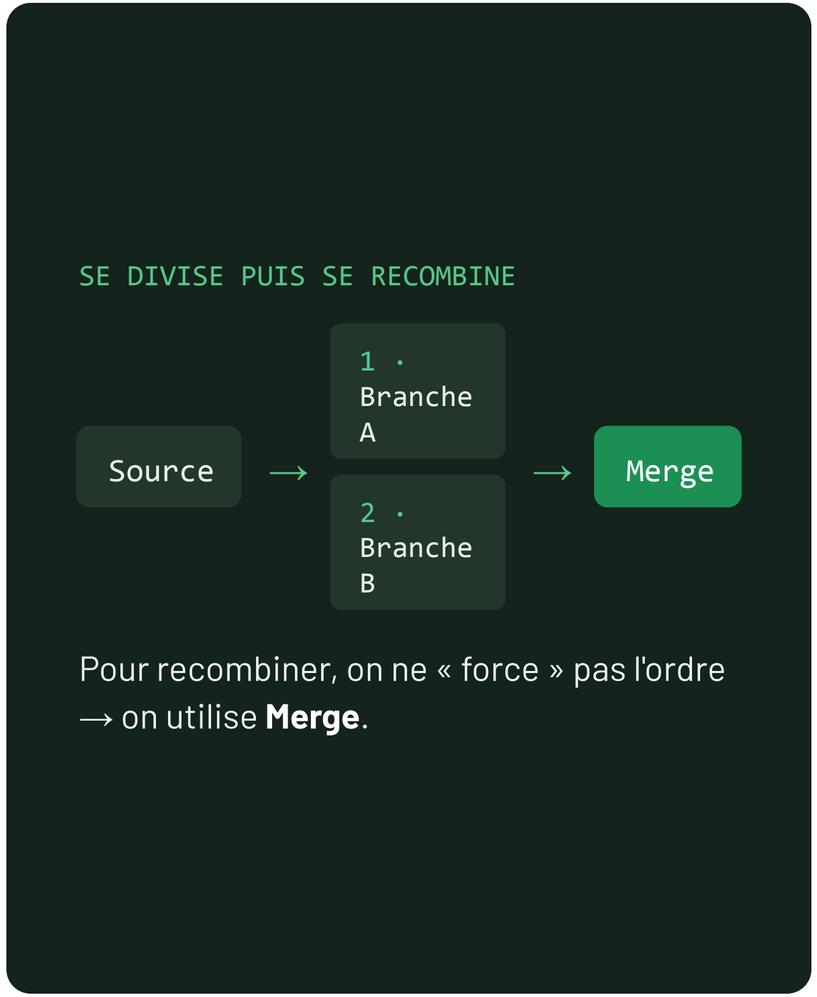
06 Découper une logique en sous-workflows réutilisables.

CADRAGE · DEUX RÉFLEXES À CORRIGER

Avant d'aller plus loin, deux idées reçues à évacuer.

01 n8n **itère déjà tout seul** : un node s'exécute une fois par item → pas besoin de boucle pour « faire la même chose sur chaque élément ».

02 L'**ordre des branches** est prévisible : chaque branche s'exécute entièrement, de haut en bas, la plus à gauche d'abord.

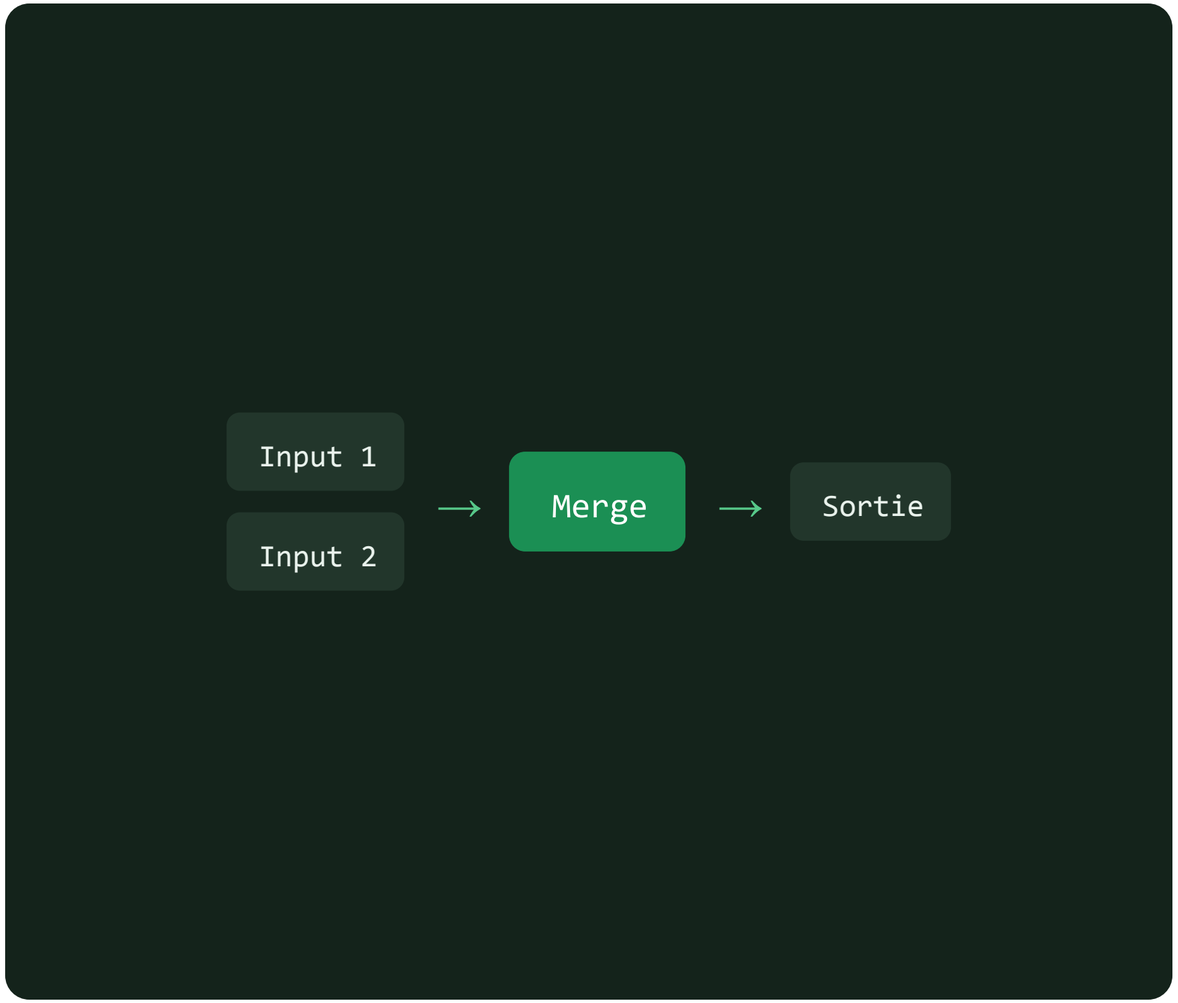


MERGE · RECOMBINER DES FLUX

Les branches divisées ne se rejoignent pas seules.

Quand un workflow se divise (après un IF, ou deux sources en parallèle), le node **Merge** les réunit en un seul flux.

Il **attend que toutes ses entrées** aient des données avant de s'exécuter.



MERGE · LES MODES

Deux façons de réunir

Append

Empile les items des entrées à la suite → une **liste plus longue** .

```
[A,B] + [C,D] → [A,B,C,D]
```

Combine · trois variantes

By Matching Fields
Jointure sur une clé — comme un JOIN SQL.

By Position
Item 1 avec item 1… la liste la plus courte l'emporte.

All Possible Combinations
Produit cartésien.

MERGE · LES PIÈGES

Trois erreurs classiques

⚠ Une branche qui **ne produit rien** bloque le Merge : il attend indéfiniment.

⚠ **Merge ≠ Aggregate** : Merge réunit plusieurs *sources* ; Aggregate regroupe des *items* d'une même source.

⚠ En mode Combine, deux champs de même nom → celui de l'**entrée 2 écrase l'entrée 1**.

ATELIER · ~15 MIN

Recombiner deux sources

01 Interroger deux API publiques (ou deux branches d'un même jeu) en parallèle.

02 Réunir les résultats avec **Append**.

03 Variante : enrichir un jeu par une clé commune avec **Combine → By Matching Fields**.

04 Observer l'effet d'une **branche vide** sur le Merge.

LOOP OVER ITEMS · QUAND (NE PAS) BOUCLER

La boucle n'est pas le réflexe par défaut.

INUTILE DE BOUCLER

n8n itère déjà : « faire la même chose sur chaque item » se fait **tout seul**.

BOUCLER EXPLICITEMENT POUR

1. **Traiter par lots** — ménager une API à quota, gros volumes.
2. **Contourner un node « exception »** qui ne traite que le 1er item.

Node : `Loop Over Items` — anciennement *Split in Batches*.

LOOP OVER ITEMS · FONCTIONNEMENT

Découper en lots, avec deux sorties

- Découpe l'entrée en **lots** d'une taille définie.
- Deux sorties : `loop` (le lot courant) et `done` (tous les lots traités).
- **S'arrête tout seul** — inutile d'ajouter un IF pour couper.

MOTIF FRÉQUENT



La sortie `done` part vers la suite quand tous les lots sont traités.

ATELIER · ~15 MIN

Traitement par lots

01 Partir d'une liste de ~20 items.

02 **Loop Over Items** (taille de lot : 5) → appel API → **Wait**.

03 Observer les sorties `loop` / `done` et l'arrêt automatique.

LE CODE NODE · QUAND Y PASSER

Quand une expression ne suffit plus.

Une expression suffit pour une ligne lisible. On passe au **Code node** pour une logique multi-lignes, l'agrégation, le dédoublonnage, une restructuration complexe.

`JavaScript` `Python`

⚠

Le Code node **ne peut pas** accéder aux fichiers ni faire de requêtes HTTP.

→ utiliser les nodes dédiés.

CODE NODE · LES DEUX MODES D'EXÉCUTION

Le choix structurant

Run Once for All Items

Le code tourne **une fois** pour tout le lot.

```
$input.all()
```

Pour agréger, trier, dédoublonner.

Run Once for Each Item

Le code tourne **une fois par item**.

```
$json
```

Pour une transformation ligne à ligne.

On renvoie toujours des **items** (objets).

CODE NODE · JAVASCRIPT (PREMIER CHOIX)

Le langage natif de n8n.

- Le plus complet et le plus **rapide**.
- Accès aux variables intégrées `$input`, `$json`, `$now`…
- Self-hosted : modules Node et paquets **npm** autorisables.

DÉDOUBLONNAGE · MODE ALL ITEMS

```
const vus = new Set(); return  $input.all().filter(item => {

const  doublon = vus.has(item.json.email);
  vus.add(item.json.email);

return  !doublon;
});
```

Résultat : `3 items → 2 items`

CODE NODE · PYTHON (SPÉCIFICITÉS)

Disponible, mais un choix particulier

- En n8n 2.x, s'exécute via un **task runner natif** (à activer, self-hosted).
- **Plus lent** que JavaScript (compilation).
- **Bibliothèques limitées** : à configurer sur le runner ; aucune sur n8n Cloud.

NOTATION D'ACCÈS AUX CHAMPS

~~`item.json.champ`~~ 1.x · Pyodide

```
item["json"]["champ"]
```
2.x · natif

Le runner natif impose les **crochets**.

CODE NODE · JAVASCRIPT OU PYTHON ?

Dans le doute → JavaScript

JAVASCRIPT · PAR DÉFAUT

Natif, rapide, intégration complète, écosystème **npm**.

PYTHON · QUAND

On a déjà du code Python, une **compétence d'équipe** , ou un besoin d'une **bibliothèque précise** déjà configurée sur le runner.

ATELIER · ~20 MIN

Logique sur-mesure

01 **JavaScript**, mode *All Items* : dédoublonner les items sur un champ (`email`).

02 **JavaScript**, mode *Each Item* : composer un champ `resume` à partir de plusieurs champs.

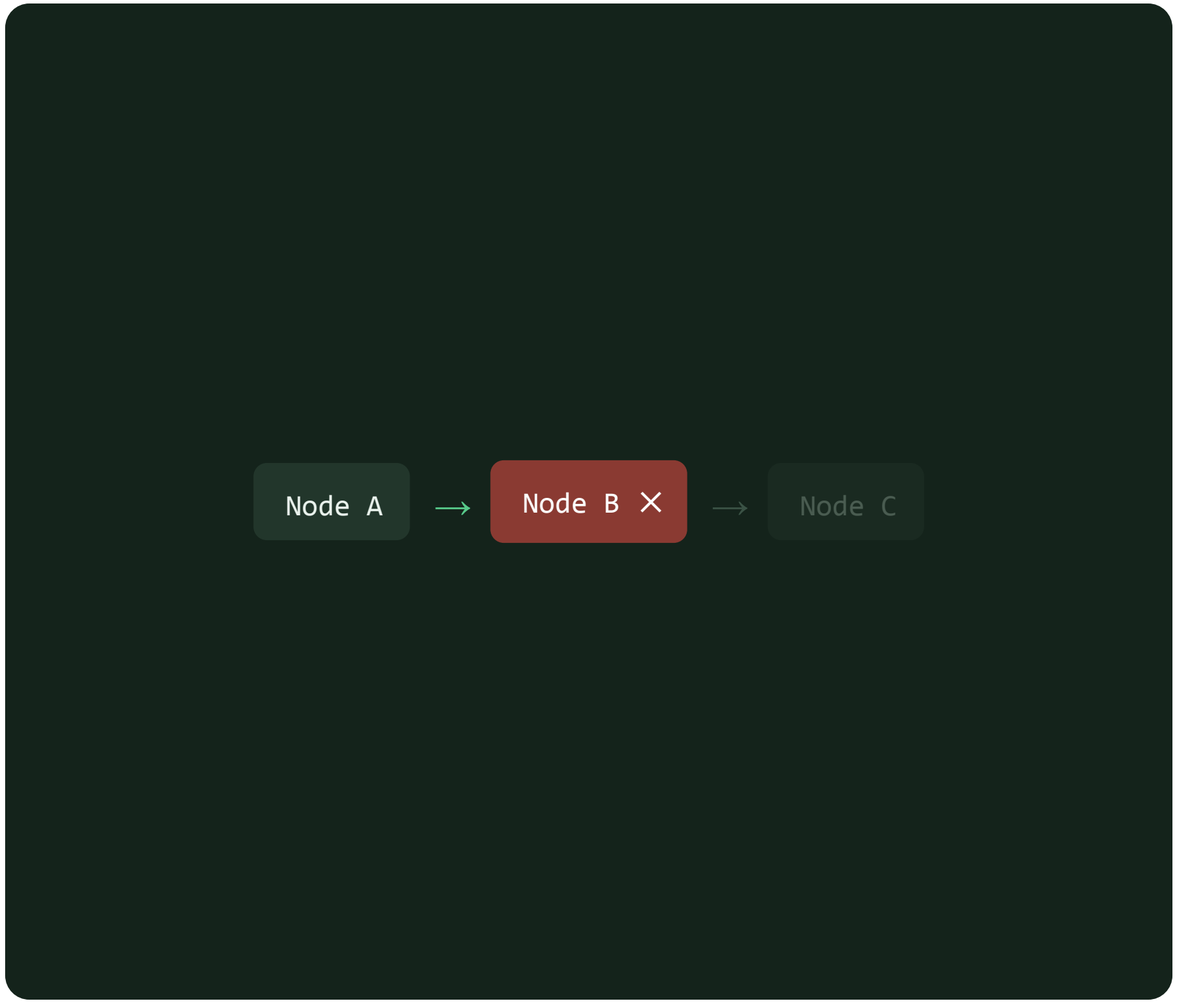
03 **Bonus** : réécrire l'étape 1 en Python (notation par crochets) et comparer.

ROBUSTESSE · LE PROBLÈME PAR DÉFAUT

Par défaut, un échec arrête tout.

Dès qu'un node échoue, l'exécution **s'arrête à ce node** .
En production, c'est insuffisant : on veut **retenter**, **isoler** l'échec, ou au moins **être alerté**.

n8n propose deux niveaux à combiner : le **node** et le **workflow**.



ROBUSTESSE · AU NIVEAU DU NODE

Deux réglages sur le node

`Retry On Fail`

Retente N fois avec un délai → pour les erreurs **transitoires** (timeout, rate limit).

`On Error` · ex-Continue On Fail

Stop Workflow (défaut)
Arrêt, exécution en erreur.

Continue
Poursuit, transmet les données d'erreur.

Continue (using error output)
Ajoute une **sortie d'erreur** dédiée sur le node.

ROBUSTESSE · AU NIVEAU DU WORKFLOW

Un filet de sécurité global

Error Trigger + Error Workflow

Un workflow dédié, déclenché **automatiquement** en cas d'échec, qui reçoit le contexte (workflow, node fautif, message) → handler centralisé (Slack / e-mail).

Stop and Error

Échouer **volontairement** avec un message clair (règle métier violée).

⚠ L'Error Trigger ne se déclenche **que** sur les exécutions de production (pas en test manuel), et pas si le node « capture » déjà l'erreur.

ROBUSTESSE · LE MODÈLE EN COUCHES

Chaque couche attrape ce que la précédente laisse passer

`Retry On Fail`	pour le **transitoire**.
`Sortie d'erreur`	pour l'**échec partiel** (certains items).
`Error Trigger global`	**filet de sécurité** pour tout le reste.
`Stop and Error`	pour les **règles métier**.

ATELIER · ~15 MIN

Rendre un workflow robuste

Sur l'appel API de l'atelier précédent :

01 Activer **Retry On Fail** (2 essais).

02 Passer en **On Error → Continue (using error output)** et router l'erreur vers un log.

03 Créer un workflow **Error Handler** (Error Trigger → notification) et le rattacher.

SOUS-WORKFLOWS · POURQUOI

Isoler une logique répétée dans une brique.

Quand une logique se répète (nettoyer, notifier, logger), on l'isole dans un **sous-workflow**.

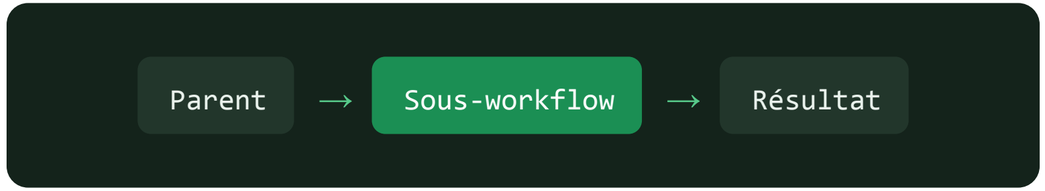
Node : `Execute Sub-workflow` — anciennement *Execute Workflow* .

Réutilisation (DRY)

Lisibilité du parent

Testabilité de la brique

Le parent envoie, la brique renvoie



⚠ Ne pas merger la sortie d'un sous-workflow avec le parent → utiliser directement sa sortie.

FIL ROUGE

Ces sous-workflows deviendront les **outils (tools) de l'agent IA** — node « Call n8n Workflow Tool ».

ATELIER · ~20 MIN

Extraire un sous-workflow

01 Reprendre une logique répétée — ex. « normaliser puis logger un item ».

02 La déplacer dans un sous-workflow, l'appeler avec **Execute Sub-workflow**.

03 Documenter les **entrées / sorties** — ce sous-workflow servira d'outil à l'agent.

CHAPITRE 3 · POINTS CLÉS À RETENIR

Cinq points clés

- ✓ **Merge** recombine des branches (Append / Combine) — attention à la branche vide.
- ✓ **Loop Over Items** seulement pour le batching et les nodes « exception ».
- ✓ **Code node** dès qu'une expression ne suffit plus ; JavaScript par défaut, Python si besoin spécifique.
- ✓ Robustesse **en couches** : Retry → sortie d'erreur → Error Trigger → Stop and Error.
- ✓ **Sous-workflows** = réutilisation… et futurs outils de l'agent IA.

EXERCICES · CHAPITRE 3

Exercice 3.1 — Router selon une condition

1. Générer une liste d'items avec un champ `montant`.
2. Avec un **IF**, séparer `montant > 1000` du reste ; nommer les deux branches.
3. Refaire avec un **Switch** à 3 tranches, puis un **Filter** qui ne garde que les gros montants.

Objectif : distinguer IF, Switch et Filter selon le besoin.

EXERCICES · CHAPITRE 3

Exercice 3.2 — Joindre deux sources

1. Deux jeux d'items partageant une clé `id` (ex. clients / commandes).
2. Les réunir avec un **Merge** en mode `By Matching Fields`.
3. Comparer avec `Append` et `By Position` : observer la différence.

Objectif : choisir le bon mode de Merge et repérer ses pièges.

EXERCICES · CHAPITRE 3

Exercice 3.3 — Dédoublonner en Code node

1. Partir d'une liste d'items contenant des `email` en double.
2. En **Run Once for All Items**, écrire un Code node qui supprime les doublons.
3. Vérifier le nombre d'items en sortie ; discuter : expression ou Code node ?

Objectif : savoir quand passer au Code node et manier `$input.all()`.

EXERCICES · CHAPITRE 3

Exercice 3.4 — Gérer les erreurs

1. Sur un node HTTP Request, activer **Retry On Fail** (2 essais).
2. Passer **On Error** sur « Continue using error output » et traiter la branche d'erreur.
3. Créer un workflow d'erreur (**Error Trigger**) qui journalise le contexte.

Objectif : appliquer la robustesse en couches : retry → sortie d'erreur → Error Trigger.

EXERCICES · CHAPITRE 3

Exercice 3.5 — Défi : isoler un sous-workflow

1. Isoler une logique récurrente (ex. « enrichir et valider un contact ») dans un nouveau workflow.
2. L'appeler depuis un parent via **Execute Sub-workflow** ; passer des données et récupérer le résultat.
3. Tester le sous-workflow isolément, puis via le parent.
4. Discuter : comment ce sous-workflow deviendra un outil d'agent IA au Jour 5.

Objectif : découper proprement une logique réutilisable et testable.